

31338 Network Servers

32520 Systems Administration

Week 1

Introduction and System Startup

Dr Litao Yu

Subject Information

- Please check Subject Outline on Canvas for details!
- Check Subject Outline -> Assessment
 - Assessment 1 - Skills Test (20%): Week 5 in Lab session
 - Assessment 2 – Lab Tests & Quizzes (30%): Weeks 3, 4, 6, 7, 8, 10 and 11
 - Assessment 3 – Final Skills test (40%): Week 12
 - Assessment 4 - Learning Journal (10%): Submitted to Canvas (**Weeks 6 & 12**)

Note: For quizzes and skill tests, you can only use Learning Journal as your reference. You can't do online search and/or read other books.

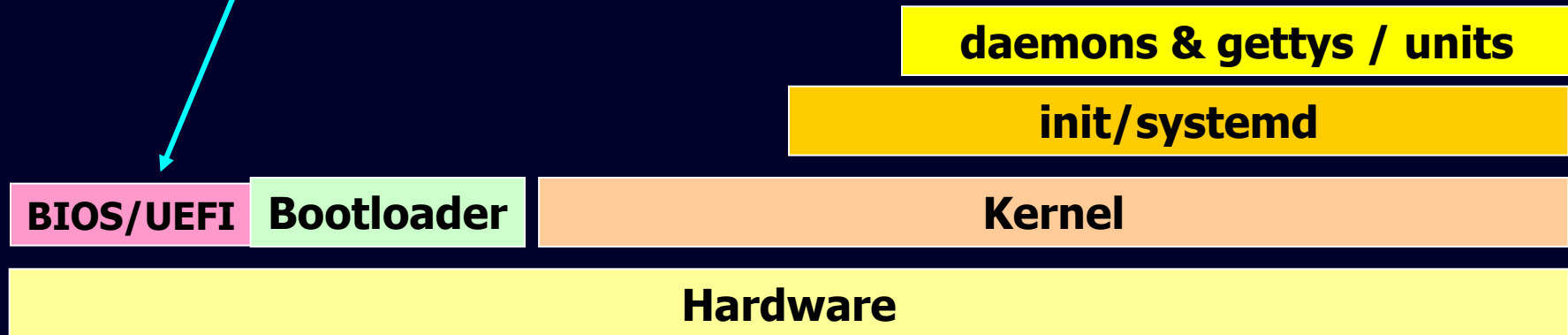
- Class Info:
 - Lectures are online only.
 - Labs are run On-Campus only in CB11.09.405.
- Slides and Labs:
 - **For labs, you are strongly required to go through the pre-recorded videos!**
 - For online lectures, you will catch up the basic concepts. Then you need to connect them to the lab practices.
 - System: Windows Laptop/PC or Lab PC with **CentOS 10 and Windows Server 2025**

Typical sysadmin tasks

- Installing & configuring servers
- Installing & configuring application software
- Creating & maintaining user accounts
- Backing up & restoring files
- Monitoring & tuning performance
- Configuring a secure system
- Using tools to monitor security
- Upgrades, security patches
- Fault finding / fixing
- Automating repetitive tasks
- Answering questions
- Negotiating & monitoring service agreements
- Meetings
- Initiating or updating policy documentation
- Purchasing new equipment
- Maintaining awareness of security threats & patches

The Boot Process

1. The server **firmware** (BIOS/UEFI) starts, performing a quick check of the hardware, called a Power On Self-Test (POST), and then it looks for a boot loader program to run from a bootable device.
2. The **boot loader** runs and determines what kernel to load.
3. The **kernel** program loads into memory; prepares the system, such as mounting the **root partition**; and then runs the initialization program.
4. The **initialization** process starts the necessary background programs required for the system to operate (such as a **graphical desktop manager** for desktops, or web and database applications for servers).

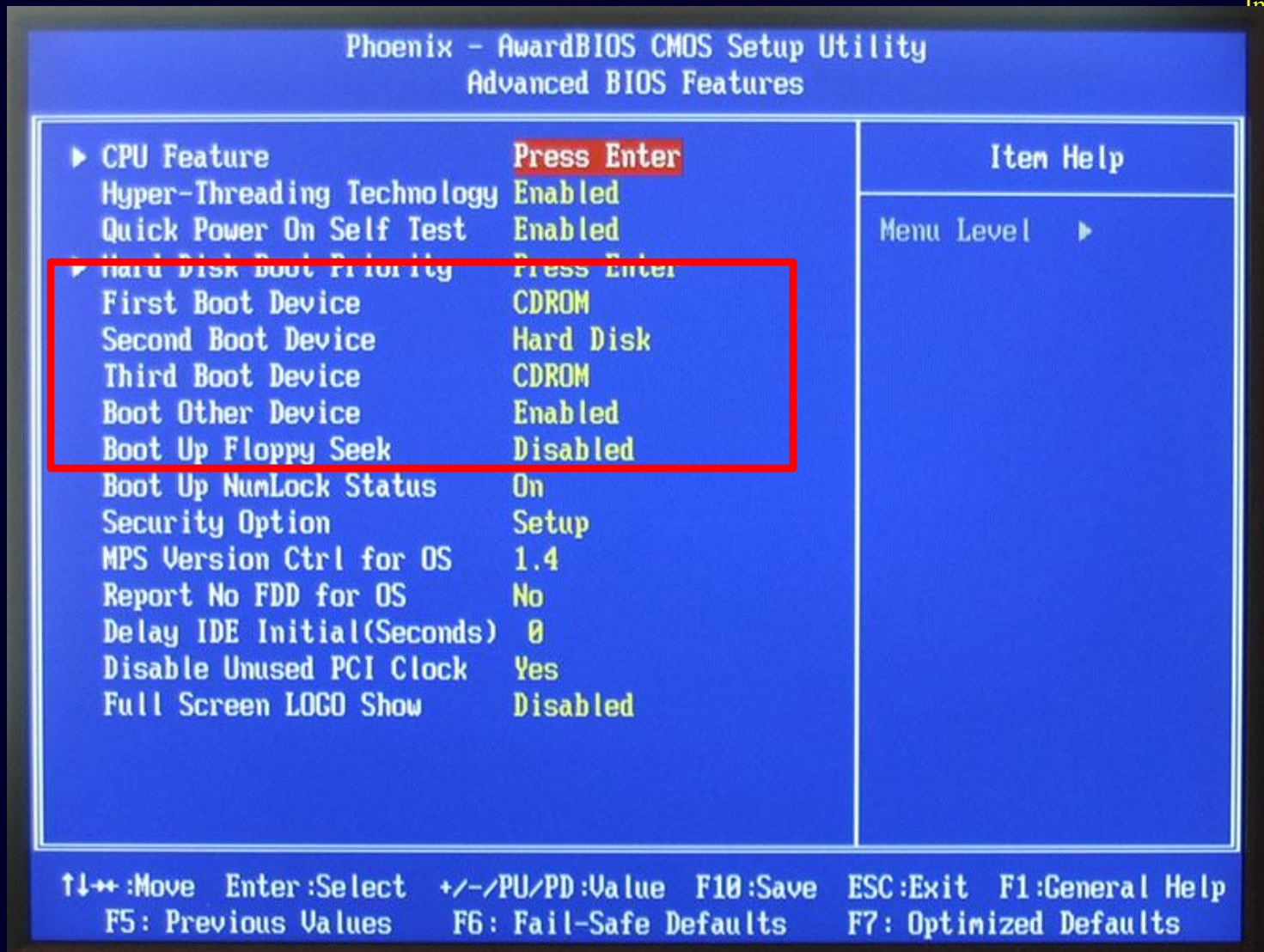


BIOS (Basic Input/Output System) and UEFI (Unified Extensible Firmware Interface) are both firmware interfaces that initialize hardware and load the operating system

Firmware: BIOS vs. UEFI

- BIOS = Basic Input/Output System
 - located on ROM chip on motherboard, NVRAM for settings.
- UEFI = the Unified Extensible Firmware Interface, to replace the BIOS
- Most basic function:
 - Execute a power-on self test (POST) and then go to a "known place" known as Master Boot Record (MBR) that is a traditional partition style in computer system and load a bootstrap program into memory and start it running.
- BIOS usually allows configuration of the "known place"
 - e.g. may be a disk sector (hard disk, floppy or CD-ROM)
 - some BIOS/ROM chips can boot from a network address

BIOS



Unified Extensible Firmware Interface (UEFI)

- Created by Intel in 1998 and became a standard in 2005
- Specify a special disk partition called EFI System Partition (ESP) to store boot loader program (can be any size)
- ESP can store multiple boot load programs for multiple OS
- ESP normally mounted in `/boot/efi` directory and boot loader files are stored as `.efi`
- Most of time, backwards compatible

Note: Extensible Firmware Interface (EFI)



To check the /sys/firmware directory and see if the efi directory is exist.
If not, using traditional BIOS to boot the computer

Boot loader

- Most basic function:
 - find the kernel and run it, a bridge between system firmware and kernel
- Boot loaders support multiple kernels
 - e.g., dual-boot machines that can boot Windows and Linux
 - e.g., Linux machines when you are installing a new kernel
- Boot loader is in Master Boot Record (MBR) for BIOS adopted, otherwise, EFI System Partition (ESP) for UEFI
- Main boot loaders in Linux on PCs:
 - LILO = Linux Loader (old, only found on ancient installations, discontinued in Dec 2015)
 - GRUB Legacy = Grand Unified Boot Loader (created in 1999)
 - **GRUB2 created in 2005 as a total rewrite of GRUB Legacy**

Boot loader – GRUB2

- GRUB2 configuration file: `grub.cfg`. Saved in `/boot/grub` or `/boot/grub2` for BIOS, and `/boot/efi/EFI/distro-name` for UEFI

- Sample GRUB 2 configuration file

[...]

```
menuentry "CentOS Linux" {
```

[...]

```
    set root=(hd1,1)
```

1st partition in the 2nd
hard drive

```
    linux16 /vmlinuz[...]
```

```
    initrd /initramfs[...]
```

```
}
```

```
menuentry "Windows" {
```

```
    set root=(hd0,1)
```

1st partition in the 1st
hard drive

[...]

Boot loader – GRUB2

- Rebuild the configuration file rather than install
 - `grub-mkconfig` Or `grub2-mkconfig`. Read configuration files in `/etc/grub.d` and assemble the commands into `grub.cfg`
 - Update the configuration file manually through
 - `grub2-mkconfig > /boot/grub2/grub.cfg`

Troubleshooting startup problems

- Boot into single user mode (described later)
- Investigate key configuration files and fix errors
- Reboot and test
- “booting into single user mode” requires interacting with grub at boot time

Single-user mode limits access to your company file to one person at a time. With more than one user license, you can switch to multi-user mode where you can log in to a company file on different computers at the same time.

Boot loader – Windows

- BOOTMGR looks for active partition
- BOOTMGR reads the BCD (boot configuration database) file from the `\boot` directory
 - The BCD contains configuration parameters (older windows used `\boot.ini`)
- BOOTMGR executes Windows Loader (**winload.exe**)
 - or winresume.exe when resuming hibernated system
 - Winloader loads drivers & transfers control to the windows kernel

<https://learn.microsoft.com/en-us/windows-hardware/drivers/bringup/boot-and-uefi>

Q&A

- What program does a system's firmware start at boot time?
 - A. A boot loader
 - B. The BIOS program
 - C. The UEFI program
 - D. The POST command
 - E. The init program

init/systemd for system boots

- `init/systemd` is the first process ever to run in a UNIX system
 - always has Process ID (PID) of 1. (hint: using `top` to check the PID)
- `Sys V init` processes instructions from `/etc/inittab`
 - typically starts system daemons & gettys
 - daemons are background processes that perform system tasks
 - gettys “get ttys” are processes that monitor terminals for activity (logins); One getty process serves one terminal.
 - may also start a graphical login screen (XDM: X Window Display Manager)
- `systemd` (2010+, ‘d’: daemon), the most popular now for SB
 - Reduce initialization time by starting services in a parallel manner
 - `which init` #find the init program file location
 - `readlink -f /usr/sbin/init` #check the init program for links
#/usr/lib/systemd/systemd in our lab

In Linux, a daemon is a background process that runs without direct user interaction, typically performing system tasks or providing services

systemd

- Unit Files **rather run a lot of daemons!**
 - A unit defines a service, a group of services, or an action. Each unit consists of a name, a type, and a configuration file.
 - Units are identified as **name.type**.
- **systemctl** utility to manage system services
 - **systemctl** COMMAND UNIT-NAME
 - COMMAND=`disable, enable, restart, start, status, stop, reload`
 - COMMAND=`is-active, is-enabled, is-failed`
 - UNIT-NAME: `firewalld, ntpd, dhcpd`
- **Category 1: Service Unit Files .service**
 - Contains information such as which environment file to use when a service must be started, located in different directories
 - **systemctl list-unit-files** #show unit file name and state (e.g., enabled, disabled, static, etc.)
 - **systemctl cat xxx.service** #show more information

systemd

- **Category 2: Target Unit Files .target**
 - Group together various services to start at system boot time.
 - `systemctl list-units` #check running units
 - `systemctl get-default` #graphical.target
 - `systemctl cat graphical.target`
 - obtain, set and jump between system targets
 - COMMAND=`get-default, set-default, isolate (jump)`
 - `poweroff.target` – shuts down and power offs the system
 - `rescue.target` – also for system recovery – requires root password
 - `multi-user.target` – multi-user mode with no graphical login
 - `emergency.target` – used for emergency system recovery
 - `graphical.target` – full graphical mode
 - `reboot.target` – shuts down and boots the system again
 - A target can be a part of another target, e.g., `graphical.target` includes `multi-user.target` which depends on `basic.target` and others
 - `systemctl list-unit-files --type=target` #overview all targets

Sys V init

- Sys V `init` processes instructions from `/etc/inittab`
- Linux (and Sys V UNIX systems) has conventionally 7 runlevels (more up to 10 in some cases)
 - A runlevel defines the state of the machine after boot.
 - Only one runlevel is executed on startup; run levels are not executed one after another.

Runlevel purposes and systemd-targets

RUNLEVEL	PURPOSE	SYSTEMD-TARGETS
Runlevel 0	Shuts down the system (halt mode)	poweroff.target
Runlevel 1	Single-user mode	rescue.target
Runlevel 2	Multiuser mode without networking (no network file system)	multi-user.target
Runlevel 3	Multiuser mode with networking (normal mode start); standard runlevel for Linux-based systems without GUI	multi-user.target
Runlevel 4	User-definable	multi-user.target
Runlevel 5	Multiuser mode with networking (supports GUI); standard runlevel for many Linux-based systems with GUI and desktop Unix systems	graphical.target
Runlevel 6	Reboots the system to restart it	reboot.target

Managing runlevels and services

Runlevel

```
grep :initdefault: /etc/inittab #Check default runlevel
runlevel #Check current runlevel (shows previous and current)
telinit 4 #Change runlevel
init 4 #Change runlevel
telinit q #Re-read /etc/inittab file, change runlevel and use runlevel
        directly to show current runlevel
```

Services

```
service SCRIPT COMMAND [OPTIONS]
        COMMAND=restart, start, status, stop, reload

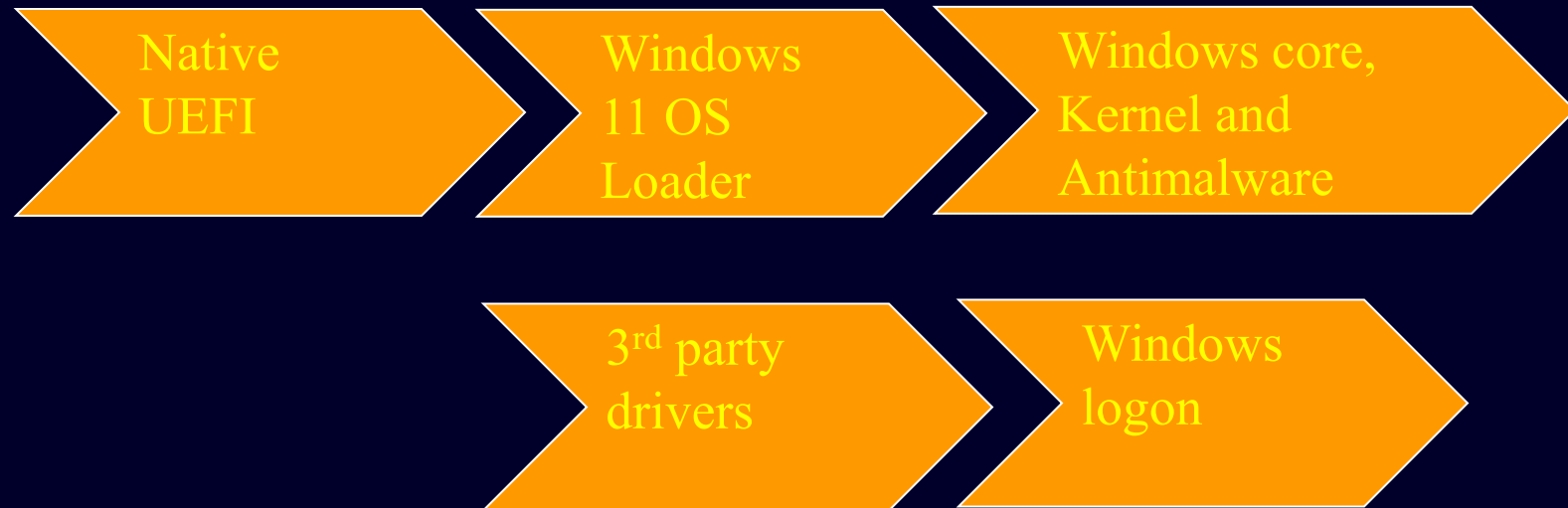
service cups start # Common UNIX Printing System to act as
a print server
service httpd start # to start a web service
```

Log files

- If something goes wrong during booting, the system logs may help diagnose the problem
- *Kernel ring buffer* – kernel and module messages
 - `dmesg | less` #dmesg: write kernel messages to standard output
 - (after boot, may be in `/var/log/dmesg` or `/var/log/boot.log`)
 - `journalctl` # viewing logs which are collected by system, use `-k` or `-dmesg` to retrieve kernel messages
- Main system log file – `/var/log/messages`
 - Contents configured through syslog daemon – more later
- Other log files in `/var/log`

Windows startup process

- Windows 11 Startup Architecture



Q&A

- Which of the following is *not* a systemd target unit?
 - A. runlevel7.target
 - B. emergency.target
 - C. graphical.target
 - D. multi-user.target
 - E. rescue.target
- What memory area does Linux use to store boot messages?
 - A. BIOS
 - B. GRUB boot loader
 - C. MBR
 - D. Initrd RAM disk
 - E. Kernel ring buffer

Software Installation

- For your own computer, you need to install VMware workstation on PC/laptop. Check the lab setup page in Canvas to install VMs.
- Strongly suggest save VM files* in **SSD portable drive**, hard drive (faster, larger), or USB3.
- A "Virtual Machine" is a pretend computer that runs as a software application on a real (physical) computer
 - It allows you to have a second (virtual) computer inside your real computer
 - "Virtualisation" of servers is touted as a significant saver of money and resources (according to the trade press)

Check the Broadcom webpage for you to install VMware. Set up as a private use.

Please read the section on Canvas – Lab setup – very important! You need to import the two VMs files (.ova) to your SSD drive